MCP Unity — Documentation v1.0.0

MCP Unity — Documentation

Version: 1.0.0 | **MCP Protocol**: 2024-11-05 | **Unity**: 6000.0+ | **License**: MIT

Table of Contents

Overview

MCP Unity is a Unity Editor plugin implementing the Model Context Protocol (MCP) to connect any AI assistant — Claude, GPT-4, Gemini, Ollama and more — to your Unity projects.

It exposes **143 tools** across **13 categories** for complete Unity Editor control: scene manipulation, asset management, UI creation, animation, physics, terrain, baking, scripting and more.

It also includes an **integrated multi-provider AI Chat panel** running directly inside the Unity Editor, with real-time tool execution and streaming responses.

Key Features

- **143 Unity tools** covering all major Editor operations
- **Dynamic category loading** — only 38 core tools loaded initially, others on demand (saves tokens)

- **Integrated Chat System** — 9 AI provider presets with real-time streaming
- **Multi-provider support** — Claude, GPT-4, Gemini, DeepSeek, Groq, Mistral, Ollama, LM Studio
- **OAuth PKCE** authentication for Anthropic
- **Drag & drop** asset/GameObject references into chat
- **Server-side TTL cache** to minimize redundant Unity API calls
- **Secure path validation** — blocks directory traversal attacks
- **Thread-safe message queue** for Unity main-thread API access
- **Request monitoring** with timing and error tracking

Architecture

```
AI Client (Claude, etc.)
    │  MCP via stdio
    ▼
Node.js Bridge (Server~/)
    │  JSON-RPC 2.0 over WebSocket
    ▼
Unity Editor Plugin (Editor/)
    │
    ├─ Tool Execution (main thread)
    └─ Chat Panel ──── Direct HTTP/SSE ───► LLM APIs
```

Components

Component	Language	Role
Node.js Bridge (`Server~/src/`)	TypeScript	MCP stdio server, forwards requests to Unity via WebSocket, TTL cache
Unity Plugin (`Editor/McpServer/`)	C#	WebSocket server inside Unity, executes tools on main thread
Chat System (`Editor/McpServer/Chat/`)	C#	Integrated multi-provider LLM chat panel

Communication Flow

1. AI client sends MCP request via **stdio** to the Node.js bridge
2. Bridge forwards over **WebSocket** (JSON-RPC 2.0) to Unity Editor
3. Unity queues the message — processes on the **main thread** (required for Unity API)
4. Unity returns the response via WebSocket back to the bridge
5. Bridge returns the MCP response to the AI client via stdio

Threading Model

WebSocket messages arrive on background threads. `McpBehavior.OnMessage` queues them to a `ConcurrentQueue<QueuedMessage>`. `EditorApplication.update` processes up to **10 messages per frame** on the main thread.

Installation

Prerequisites

Requirement	Version
Unity	6000.0+ (Unity 6)
Node.js	18+
npm	9+

Option A — Unity Package Manager (Git URL)

In Unity: **Window** > **Package Manager** > **+** > **Add package from git URL**:

```
https://github.com/JulianKerignard/mcp-unity.git
```

Option B — Local Installation

Copy the package into your project's `Packages/` folder:

```
YourProject/
└── Packages/
    └── com.juliank.mcp-unity/
        ├── Editor/
        ├── Plugins/
        ├── Server~/
        └── package.json
```

Build the Node.js Bridge

```
cd Packages/com.juliank.mcp-unity/Server~/
npm install
npm run build
```

> **Tip**: Use the **Setup Wizard** (`Tools > MCP Unity > Setup Wizard`) to do this automatically.

Configure Claude Desktop

Add to `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS):

```json
{
  "mcpServers": {
    "mcp-unity": {
      "command": "node",
      "args": ["/absolute/path/to/Server~/build/index.js"],
      "env": { "UNITY_PORT": "8090" }
    }
  }
}
```

Configure Claude Code CLI

```
claude mcp add mcp-unity -- node /absolute/path/to/Server~/build/
        index.js
```

Environment Variables

Variable	Default	Description
`UNITY_HOST`	`localhost`	Unity WebSocket host
`UNITY_PORT`	`8090`	Unity WebSocket port
`DEBUG`	`false`	Enable debug logging (`true` or `1`)
`REQUEST_TIMEOUT`	`10000`	Request timeout in ms
`RECONNECT_INTERVAL`	`3000`	Reconnect interval in ms
`MAX_RECONNECT_ATTEMPTS`	`3`	Max reconnection attempts

Start the Server

In Unity: **Tools** > **MCP Unity** > **Server Window** → click **Start Server**.

The status indicator turns **green** when ready.

Setup Wizard

The **Setup Wizard** opens automatically the first time the package is imported. It guides you through:

1. **Node.js check** — verifies Node.js 18+ is installed
2. **Build bridge** — runs `npm install && npm run build` automatically
3. **Claude config** — generates and copies the JSON config to clipboard

Re-open at any time: **Tools** > **MCP Unity** > **Setup Wizard**

Configuration

Settings are stored in `ProjectSettings/McpUnitySettings.json` and auto-saved on every change.

Setting	Default	Description
Port	8090	WebSocket server port
AutoStartServer	true	Start server on Editor load
ShowNotifications	true	Show notification popups
RequestTimeoutMs	30000	Request timeout (ms)
LogToConsole	true	Mirror logs to Unity Console
LogToFile	false	Write logs to file
MinimumLogLevel	Info	Minimum log level
MaxLogEntries	500	Max entries in log buffer
UseCustomServerPath	false	Use custom bridge path
CustomServerPath	—	Custom path to `index.js`

Tools Reference

Dynamic Tool Loading

MCP Unity uses **category-based dynamic loading** to minimize token usage:

- **38 core tools** are always available
- **105 additional tools** are loaded on demand per category
- Use `unity_list_tool_categories` to see available categories
- Use `unity_enable_tool_category` to load a category

CORE — Always Active (38 tools)

Meta-tools

Tool	Description
`unity_list_tool_categories`	List all categories with tool counts and enabled status
`unity_enable_tool_category`	Enable one or more categories to load their tools

Editor State & Selection

Tool	Description
`unity_get_editor_state`	

Tool	Description
	Get current editor state (play mode, compilation, selection)
`unity_get_selection`	Get currently selected objects
`unity_set_selection`	Set editor selection (GameObjects or assets)
`unity_get_console_logs`	Get Unity console logs with filtering
`unity_clear_console`	Clear the Unity console
`unity_take_screenshot`	Capture Scene/Game view screenshot (use `returnBase64=false` to save tokens)

Editor Workflow

Tool	Description
`unity_execute_menu_item`	Execute an allowlisted Unity menu item
`unity_run_tests`	Run Unity Test Framework tests (EditMode or PlayMode)
`unity_undo`	Perform undo or redo operations
`unity_refresh_and_compile`	Refresh AssetDatabase and trigger script re-compilation

GameObject

Tool	Description
`unity_list_gameobjects`	List scene hierarchy — use `outputMode='tree'` (90% fewer tokens)
`unity_create_gameobject_batch`	Create multiple GameObjects in one call with Undo grouping
`unity_delete_gameobject`	Delete a GameObject (finds inactive objects too)
`unity_rename_gameobject`	Rename a GameObject
`unity_set_parent`	Re-parent a GameObject (`worldPositionStays` supported)
`unity_duplicate_gameobject`	Duplicate a GameObject
`unity_move_gameobject`	Change sibling index within parent
`unity_find_gameobjects_by_component`	Find all GameObjects with a specific component
`unity_set_gameobject_active`	Activate or deactivate a GameObject

Component

Tool	Description
`unity_get_component`	Get component properties via reflection

Tool	Description
unity_add_component	Add a component with optional initial properties
unity_modify_component_batch	Modify components on multiple GameObjects in one call
unity_list_project_scripts	List all MonoBehaviour scripts in the project
unity_remove_component	Remove a component (Transform protected)

Scene

Tool	Description
unity_get_scene_info	Get current scene information
unity_load_scene	Load a scene (Single or Additive)
unity_save_scene	Save current scene or all open scenes
unity_create_scene	Create a new scene (auto-saves to Assets/Scenes/)

Script

Tool	Description
unity_create_script	Create a C# script from template (MonoBehaviour, ScriptableObject, EditorWindow)
unity_read_script	Read the contents of a C# script file
unity_get_script_info	Get reflection info (fields, properties, methods) for a type
unity_write_script	Write a complete C# script file (with backup)
unity_update_script	Find-and-replace a unique section in a script (with backup)

Memory Cache

Tool	Description
unity_memory_get	Get cached project data (assets, scenes, hierarchy, operations)
unity_memory_refresh	Refresh a cache section
unity_memory_clear	Clear cache sections

ASSET (12 tools)

Tool	Description
unity_search_assets	

Tool	Description
	Search assets using Unity filter syntax (`t:Texture`, `l:Label`, name patterns)
`unity_get_asset_info`	Get detailed asset metadata (GUID, type, size, dependencies)
`unity_list_folders`	List project folder structure
`unity_list_folder_contents`	List assets in a folder with optional type filtering
`unity_get_asset_preview`	Get asset thumbnail (use `size='small'` for fewer tokens)
`unity_get_import_settings`	Get asset import settings
`unity_set_import_settings`	Modify asset import settings
`unity_instantiate_prefab`	Instantiate a prefab in the scene
`unity_create_prefab`	Create a prefab from a scene GameObject
`unity_unpack_prefab`	Unpack a prefab instance
`unity_apply_prefab_overrides`	Apply instance overrides back to the source prefab
`unity_revert_prefab_overrides`	Revert instance overrides to source prefab values

MATERIAL (3 tools)

Tool	Description
`unity_get_material`	Get material properties (from asset path or renderer)
`unity_set_material`	Modify material properties — auto-maps for URP/HDRP/Built-in
`unity_create_material`	Create a new material with pipeline-appropriate shader

UI (9 tools)

Tool	Description
`unity_create_canvas`	Create Canvas with EventSystem (ScreenSpaceOverlay, Camera, WorldSpace)
`unity_create_ui_element`	Create UI element (Panel, Button, Text, Image, RawImage, Slider, Toggle, InputField, Dropdown, ScrollView)
`unity_get_ui_hierarchy`	Inspect UI element tree
`unity_modify_ui_element`	Modify text, color, fontSize, interactable, value, sprite, placeholder, options
`unity_set_rect_transform`	Configure RectTransform anchors, pivot, size

Tool	Description
unity_add_layout_group	Add layout group (Vertical, Horizontal, Grid)
unity_add_content_size_fitter	Add ContentSizeFitter
unity_add_layout_element	Add LayoutElement
unity_set_canvas_scaler	Configure CanvasScaler

ANIMATOR (16 tools)

Tool	Description
unity_get_animator_controller	Get controller info (states, parameters, layers)
unity_get_animator_parameters	List all parameters
unity_set_animator_parameter	Set a parameter value at runtime
unity_add_animator_parameter	Add a new parameter (Float, Int, Bool, Trigger)
unity_validate_animator	Detect issues (unreachable states, missing clips)
unity_get_animator_flow	Get full state machine flow diagram
unity_add_animator_state	Add a state to a layer
unity_delete_animator_state	Remove a state
unity_modify_animator_state	Edit state properties (speed, tag, motion)
unity_create_blend_tree	Create a Blend Tree state
unity_add_blend_motion	Add a motion to a Blend Tree
unity_add_animator_transition	Add a transition with conditions
unity_delete_animator_transition	Remove a transition
unity_modify_transition	Edit transition conditions and settings
unity_list_animation_clips	List animation clips in the project
unity_get_clip_info	Get clip details (length, frame rate, events)

TERRAIN (16 tools)

Tool	Description
unity_create_terrain	Create a Terrain with TerrainData asset
unity_get_terrain_info	Get terrain info (size, layers, trees, neighbors)
unity_modify_terrain	Modify terrain settings (pixel error, distances, rendering)

Tool	Description
unity_set_terrain_heights_batch	Sculpt heightmap: flatten, raise, lower, set, noise, smooth
unity_list_terrain_brushes	List available terrain brushes
unity_add_terrain_layer	Add a texture layer (diffuse, normal, tile size, metallic)
unity_paint_terrain_texture_batch	Paint texture layer on a region (alphamap blending)
unity_add_terrain_trees	Place trees (explicit positions or random scatter with seed)
unity_remove_terrain_trees	Remove trees from a region
unity_list_terrain_trees	List tree prototypes
unity_add_terrain_detail	Add detail (grass, mesh) to terrain
unity_paint_terrain_detail	Paint detail density on terrain
unity_remove_terrain_detail	Remove detail from a region
unity_import_heightmap	Import heightmap from PNG/RAW file
unity_export_heightmap	Export heightmap to PNG/RAW file
unity_set_terrain_neighbors	Set neighboring terrains for seamless edges

---

PHYSICS (8 tools)

Tool	Description
unity_raycast	Physics raycast — returns all hits sorted by distance
unity_setup_rigidbody	Add or configure a Rigidbody (mass, drag, constraints)
unity_setup_collider	Add collider (Box, Sphere, Capsule, Mesh, auto-detect)
unity_set_physics_material	Create and assign a PhysicsMaterial
unity_bake_navmesh	Bake navigation mesh
unity_clear_navmesh	Clear all NavMesh data
unity_get_navmesh_settings	Get agent types and area info
unity_set_navigation_static	Mark GameObjects as Navigation Static

---

AUDIO (3 tools)

Tool	Description
unity_setup_audio_source	Add/configure AudioSource with clip and mixer group

Tool	Description
unity_create_audio_mixer	Create an AudioMixer asset
unity_get_audio_mixer	Get mixer info (groups, exposed parameters)

RENDERING (13 tools)

Tool	Description
unity_configure_camera	Configure camera properties (FOV, near/far, background)
unity_render_camera_to_file	Render camera view to a PNG/JPG file
unity_get_render_pipeline_info	Get active render pipeline info (URP/HDRP/Built-in)
unity_bake_lighting	Bake lightmaps (synchronous)
unity_bake_lighting_async	Start async lightmap bake
unity_get_bake_status	Get current bake progress
unity_cancel_bake	Cancel the active bake
unity_clear_baked_data	Clear all baked lighting data
unity_get_lightmap_settings	Get lightmap settings
unity_set_lightmap_settings	Modify lightmap settings
unity_bake_occlusion	Bake occlusion culling
unity_clear_occlusion	Clear occlusion data
unity_bake_reflection_probes	Bake reflection probes

BUILD (6 tools)

Tool	Description
unity_get_build_settings	Get build configuration (target, scenes, scripting backend)
unity_manage_build_scenes	Add, remove, enable, disable, or reorder build scenes
unity_switch_platform	Switch build target (Windows, Mac, Linux, iOS, Android, WebGL)
unity_list_packages	List installed Unity packages
unity_add_package	Add a package via UPM
unity_remove_package	Remove a package via UPM

SETTINGS (11 tools)

Tool	Description
`unity_get_project_settings`	Read project settings
`unity_set_project_settings`	Modify project settings
`unity_set_quality_level`	Set quality preset
`unity_get_physics_layer_collision`	Get physics layer collision matrix
`unity_set_physics_layer_collision`	Set physics layer collision rules
`unity_list_tags`	List all tags
`unity_list_layers`	List all layers
`unity_set_tag`	Assign a tag to a GameObject
`unity_set_layer`	Assign a layer to a GameObject
`unity_create_tag`	Create a new tag
`unity_create_layer`	Create a new layer

INPUT (3 tools)

Tool	Description
`unity_get_input_actions`	Get Input Action Asset contents
`unity_add_input_action`	Add a new Input Action
`unity_add_input_binding`	Add a binding to an Input Action

ADVANCED (5 tools)

Tool	Description
`unity_set_reference`	Set an object reference on a SerializedField
`unity_set_reference_array`	Set an array of object references on a SerializedField
`unity_create_scriptable_object`	Create a ScriptableObject asset
`unity_list_scriptable_object_types`	List available ScriptableObject types in project
`unity_modify_scriptable_object`	Modify ScriptableObject properties

Resources

MCP Resources provide read-only access to Unity project state. No tool call needed.

URI	MIME Type	Description
`unity://project/settings`	`application/json`	Project name, version, platform, scripting backend
`unity://scene/hierarchy`	`application/json`	Current scene root objects with components
`unity://console/logs`	`application/json`	Recent Unity console log entries
`workflows://core`	`text/markdown`	Core workflow guide and token optimization tips
`workflows://animator`	`text/markdown`	Animator Controller complete workflow
`workflows://materials`	`text/markdown`	Materials and shaders workflow (URP/HDRP auto-detection)
`workflows://prefabs`	`text/markdown`	Prefab creation, instantiation and override workflow
`workflows://assets`	`text/markdown`	Asset search syntax and browser workflow
`workflows://terrain`	`text/markdown`	Terrain sculpting, painting and brush guide

Editor Window

Access via **Tools** > **MCP Unity** > **Server Window**. Three tabs:

Chat Tab

The primary tab — full multi-provider LLM chat panel:

- **Message input** with drag & drop asset/GameObject references
- **SSE streaming** with token-by-token display
- **Automatic tool execution** — multi-turn loop (max 10 iterations per response)
- **Markdown rendering** — code blocks, headers, tables, blockquotes, links, lists
- **Context bar** — shows token usage vs. model context window
- **Export** — Markdown, JSON, Plain Text, or clipboard

Settings Tab

Four foldout sections:

Section	Contents
Provider Settings	Active provider selector, API key, model selector, auth mode (API Key / OAuth), endpoint override
Tool Categories	

Section	Contents
	13 category toggles, preset buttons (All / Core / None), enabled count
Server Settings	Port, auto-start, notifications, timeout, logging
Advanced	Custom bridge path, bridge health indicator

Diagnostics Tab

Three foldout sections:

Section	Contents
Request Monitor	Live stats, per-request timing (tool name, duration, success/error)
Logs	Color-coded entries (debug/info/warning/error), level filter, clear
Claude Config	`.mcp.json` status, Claude Desktop config generator, copy to clipboard

Toolbar Overlay

Scene view toolbar shows MCP server status indicator and quick start/stop button.

Integrated Chat System

The Chat System is built with Unity IMGUI. It runs entirely inside the Editor without the Node.js bridge — it calls tools via the in-process `McpToolRegistry`.

Providers (9 presets)

Provider	Example Models	Context	Auth
Anthropic Claude	Sonnet 4.6, Opus 4.6, Haiku 4.5	200K	API Key / OAuth PKCE
OpenAI	GPT-4o, o3 Mini, GPT-4.1	128K	API Key
Google Gemini	2.5 Pro, 2.5 Flash	1M	API Key
DeepSeek	Chat V3.2, Reasoner R1	128K	API Key
Groq	Llama 3.3 70B, Qwen3 32B	131K	API Key
Mistral AI	Large 3, Codestral	128K	API Key
Ollama	Llama 3.3, Qwen 2.5 Coder (local)	131K	None
LM Studio	Any local model	131K	None

Provider	Example Models	Context	Auth
Custom	Any OpenAI-compatible endpoint	128K	API Key

Tool Execution Loop

1. AI response arrives via SSE
2. Parser detects `tool_use` content blocks
3. Each tool is executed on the Unity main thread via `McpToolRegistry`
4. Tool results are added to conversation and a follow-up request is made
5. Loop continues until no more tool calls or max iterations (10) reached

Drag & Drop References

Drag assets from Project window or GameObjects from Hierarchy into the chat input:

- Asset chips appear above the input (icon + name + remove button)
- `@Name` mention is inserted into the text
- On send, full context is resolved (components, asset info, dependencies)

Conversation Export

Click the ⇩ toolbar button:

Format	Description
Markdown (`.md`)	Speaker headers, fenced code blocks for tool calls, timestamps
JSON (`.json`)	Structured with provider, model, token counts, messages array
Plain Text (`.txt`)	Stripped formatting with timestamps
Clipboard	Markdown format

Authentication

- **API Keys** — per-provider, stored in `EditorPrefs` (never committed to version control)
- **OAuth PKCE** (Anthropic only) — full browser OAuth2 flow with automatic token refresh

---

Bridge Caching

The Node.js bridge caches read-only tool results to reduce Unity round-trips:

Category	TTL	Cached Tools
`editorState`	5s	`unity_get_editor_state`

Category	TTL	Cached Tools
`hierarchy`	30s	`unity_list_gameobjects`
`components`	1 min	`unity_get_component`, `unity_get_material`
`assets`	5 min	`unity_search_assets`, `unity_get_asset_info`, `unity_list_folders`
`scenes`	5 min	`unity_get_scene_info`, `unity_get_build_settings`

Write operations automatically invalidate relevant cache entries.

Development Guide

Adding a New Tool

1. C# — Register in `Editor/McpServer/Tools/`:

```
// In a partial class file under Tools/
static partial void RegisterMyTools()
{
    _toolRegistry.RegisterTool(new McpToolDefinition
    {
        name = "unity_my_tool",
        description = "Short description (keep under 80 chars)",
        inputSchema = new McpInputSchema
        {
            type = "object",
            properties = new Dictionary<string, McpPropertySchema>
            {
                ["path"] = new McpPropertySchema { type =
        "string", description = "..." }
            },
            required = new List<string> { "path" }
        }
    }, MyToolHandler);
}

private static McpToolResult MyToolHandler(Dictionary<string,
        object> args)
{
    var path = ArgumentParser.RequireString(args, "path", out var
        error);
    if (path == null) return McpToolResult.Error(error);

    // ... implementation ...
```

```
    return McpResponse.Success(new { result = "ok" });
}
```

2. Declare and call the partial method in `McpUnityServer.cs`.

3. TypeScript — Add to `Server~/src/tools.ts`:

```
{
  name: 'unity_my_tool',
  description: 'Short description',
  inputSchema: {
    type: 'object',
    properties: { path: { type: 'string' } },
    required: ['path'],
  },
  defer_loading: true, // always, unless it's a core tool
},
```

4. Add cache invalidation in `Server~/src/cache.ts` (if the tool writes state):

```
unity_my_tool: ['hierarchy', 'components'],
```

5. Rebuild: `npm run build` in `Server~/`

Conventions

Rule	Detail
Tool naming	`unity_` prefix + `snake_case`
Argument extraction	Always use `ArgumentParser` — never access dict keys directly
Serialization	Always use `JsonHelper.ToJson()` — never `JsonUtility.ToJson()` on Dictionaries
Path security	Always call `SanitizePath()` — blocks `..` traversal, enforces `Assets/` prefix
Response	`McpResponse.Success(data)` or `McpToolResult.Error(message)`
Descriptions	Keep under 80 characters (token budget: ~1200 total for all tool descriptions)

Key Files

```
Editor/McpServer/
├── McpUnityServer.cs          # Main partial class — server
lifecycle, message queue
├── McpJsonRpc.cs              # JSON-RPC 2.0 dispatcher + custom
JSON parser
├── McpToolRegistry.cs         # Tool registration, category
management, execution
├── McpResourceRegistry.cs     # Resource registration
├── McpProtocol.cs             # Protocol data classes
```

```
(JsonRpcRequest, McpContent, etc.)
├── McpSettings.cs           # Persistent settings
(ProjectSettings/McpUnitySettings.json)
├── McpDebug.cs              # Conditional logging
├── McpSetupWizard.cs        # One-time setup wizard (auto-opens
on first import)
├── McpNodeCheck.cs          # Node.js availability check at
startup
├── Tools/                   # 143 tool implementations (43 files,
partial classes)
├── Helpers/                 # ArgumentParser, ColorParser,
GameObjectHelpers, etc.
├── Models/                  # LogEntry, QueuedMessage,
MemoryCacheData
└── Utils/                   # McpConstants, PathValidator,
TypeConverter

Server~/src/
├── index.ts                 # MCP stdio server, request handlers,
TTL cache
├── UnityBridge.ts           # WebSocket client with auto-
reconnect
├── tools.ts                 # Tool definitions (38 core +
fallback)
├── resources.ts             # Resource definitions + workflow
docs
├── cache.ts                 # TTL cache + invalidation map
├── types.ts                 # Zod schemas, error codes,
BridgeConfig
└── __tests__/               # 136 vitest tests
```

Troubleshooting

Bridge & Server

Problem	Solution
Bridge won't connect	Ensure Unity is running and MCP server is started (green indicator in toolbar)
Port already in use	Change port in Settings tab → Server Settings
Tools return "not connected"	Bridge connects async — wait a few seconds after Editor starts
`JsonUtility` serialization returns `{}`	Use `JsonHelper.ToJson()` — handles `Dictionary<string, object>`
Path rejected by `SanitizePath`	Paths must start with `Assets/` and must not contain `..`
Compilation errors after script edit	Use `unity_refresh_and_compile` to trigger domain reload

Problem	Solution
Cache returns stale data	Tool should be listed in `cacheInvalidators` in `cache.ts`
WebSocket timeout	Increase `REQUEST_TIMEOUT` env var (default: 10s)
Node.js not found	Run the Setup Wizard: Tools > MCP Unity > Setup Wizard

Chat System

Problem	Solution
"No API key"	Enter key in Settings tab → Provider Settings
Tool not available in chat	Check Settings → Tool Categories — the category may be disabled
Streaming stops mid-response	Check network; press Escape then retry. Context window may be full
OAuth login fails	Ensure browser can reach Anthropic's auth server; disable popup blockers
Context fills too fast	Disable unused tool categories; use Clear to reset conversation
Drag & drop not working	Drop on the input field area, not the message area

Common Error Messages

Error	Cause	Fix
`GameObject not found: X`	Object is inactive or path is wrong	Use `unity_list_gameobjects` to verify; tool searches inactive objects too
`Required parameter 'X' is missing`	Tool called without a required argument	Check the tool's `inputSchema.required` fields
`Tool 'X' exists but category 'Y' is not enabled`	Category not loaded	Call `unity_enable_tool_category` with the category name
`Invalid asset path`	Path contains `..` or doesn't start with `Assets/`	Use full `Assets/...` paths